Univerza v Ljubljani
Fakulteta za računalništvo in informatiko

Jan Zajc

Svoboda na dotik: Ali je popolnoma odprtokoden telefon možen?

OSNUTEK DIPLOMSKE NALOGE

VISOKOŠOLSKI STROKOVNI ŠTUDIJSKI PROGRAM
PRVE STOPNJE
RAČUNALNIŠTVO IN INFORMATIKA

Mentor: viš. pred. dr. David Jelenc

Ljubljana, 2026

To delo je ponujeno pod licenco *Creative Commons Priznanje avtorstva-Deljenje pod enakimi pogoji 2.5 Slovenija* (ali novejšo različico). To pomeni, da se tako besedilo, slike, grafi in druge sestavine dela kot tudi rezultati diplomskega dela lahko prosto distribuirajo, reproducirajo, uporabljajo, priobčujejo javnosti in predelujejo, pod pogojem, da se jasno in vidno navede avtorja in naslov tega dela in da se v primeru spremembe, preoblikovanja ali uporabe tega dela v svojem delu, lahko distribuira predelava le pod licenco, ki je enaka tej. Podrobnosti licence so dostopne na spletni strani creativecommons.si ali na Inštitutu za intelektualno lastnino, Streliška 1, 1000 Ljubljana.

Izvorna koda diplomskega dela, njeni rezultati in v ta namen razvita programska oprema je ponujena pod licenco GNU General Public License, različica 3 (ali novejša). To pomeni, da se lahko prosto distribuira in/ali predeluje pod njenimi pogoji. Podrobnosti licence so dostopne na spletni strani `http://www.gnu.org/licenses/`.

Besedilo je oblikovano z urejevalnikom besedil LaTeX.

Kandidat: Jan Zajc
Naslov: Svoboda na dotik: Ali je popolnoma odprtokoden telefon možen?
Vrsta naloge: Diplomska naloga na visokošolskem strokovnem programu prve stopnje Računalništvo in informatika
Mentor: viš. pred. dr. David Jelenc

Opis:
Opis diplome

Title: Freedom at Your Fingertips: Is a Fully Open-Source Phone Possible?

Description:
Opis diplome v angleščini

Kazalo

Povzetek

Abstract

1 Uvod **1**

2 Pregled področja **3**

2.1 Duopol Android/iOS in posledice za uporabnike 3

2.2 Sistem iOS 4

2.3 Sistem Android 4

2.4 Akterji mobilnega ekosistema 5

2.5 Alternativni Android ROM-i 6

2.6 Mobilni Linux: postmarketOS, Ubuntu Touch in sorodni projekti 7

2.7 Namenske Linux naprave 8

2.8 Povzetek 9

3 Teoretična izhodišča **11**

3.1 Odprtokodna in prosta programska oprema 11

3.2 Arhitektura sistema: od zagona do uporabniškega prostora . . 13

3.3 Gonilniki, strojna programska oprema in binarni dodatki . . . 15

3.4 Odprtost, varnost in zasebnost 16

3.5 Vzdrževanje in življenjski cikel naprav 18

4 Model "popolnoma odprtokodnega telefona" 19

4.1 Idealni referenčni model 20

4.2 Slojni pogled na odprtost telefona 20

4.3 Kriteriji odprtosti po slojih 21

4.4 Kriteriji praktične uporabnosti 25

4.5 Način ocenjevanja 26

4.6 Povzetek 28

5 Analiza trenutnega stanja odprtokodnih mobilnih rešitev 29

5.1 Alternativni ROM-i za sistem Android 30

5.2 Mobilne distribucije sistema GNU/Linux na obstoječi strojni opremi za sistem Android 36

Literatura 43

Povzetek

Naslov: Svoboda na dotik: Ali je popolnoma odprtokoden telefon možen?

Avtor: Jan Zajc

placeholder

Ključne besede: odprtokodni telefon, odprtokodna programska oprema, pametni telefoni, mobilni operacijski sistemi, zasebnost.

Abstract

Title: Freedom at Your Fingertips: Is a Fully Open-Source Phone Possible?

Author: Jan Zajc

placeholder

Keywords: open-source phone, open-source software, smartphones, mobile operating systems, privacy.

Poglavje 1

Uvod

placeholder

Poglavje 2

Pregled področja

Mobilni telefoni so se iz komunikacijskih naprav razvili v splošnonamenska računalniška orodja, ki jih uporabljamo za delo, zabavo, navigacijo, avtentikacijo in komunikacijo. V praksi to pomeni, da je operacijski sistem pametnega telefona postal ena izmed ključnih komponent digitalnega življenja uporabnika. Trenutno stanje na trgu je močno centralizirano. Mobilni ekosistem je v veliki meri razdeljen med Android in iOS, kar ima neposredne posledice za svobodo uporabnika, zasebnost in življenjsko dobo naprav. V tem poglavju najprej opišemo ključne lastnosti obeh prevladujočih platform, nato pa pregledamo glavne odprtokodne pristope in njihove omejitve.

2.1 Duopol Android/iOS in posledice za uporabnike

Android in iOS predstavljata praktično celoten trg mobilnih operacijskih sistemov. Po podatkih StatCounter (globalni delež glede na uporabo) Android dosega približno 68 %, iOS pa približno 31 % [50]. Podobno sliko kažejo tudi tržne analize prodajnih deležev [51]. Ne glede na natančne odstotke je ključna ugotovitev, da sta v praksi relevantni skoraj izključno ti dve platformi.

Takšna koncentracija moči pomeni, da sta Google in Apple posrednika med uporabnikom in napravo. Določata, kako se namešča programska oprema,

kakšna so pravila za aplikacije, kakšni so varnostni mehanizmi in katere funkcionalnosti so sploh dostopne. Čeprav sta varnost in zanesljivost pomembni, tak model pogosto privede do tega, da se nadzor uporabnika nad napravo (npr. prilagajanje sistema in izbira storitev) podredi poslovnim interesom ekosistema.

2.2 Sistem iOS

Sistem iOS je zasnovan kot strogo nadzorovan sistem. Uporabnik praviloma namešča aplikacije prek uradne Applove trgovine z aplikacijami (App Store), sistem pa je zgrajen okoli modela podpisovanja kode in preverjanja celovitosti [13, 14]. Tak pristop ima prednosti, kot so enotna uporabniška izkušnja, relativno predvidljiv varnostni model in dolga programska podpora za naprave. Hkrati pa pomeni tudi, da ima uporabnik omejen vpliv na to, kaj se lahko poganja na napravi in kakšne posege v sistem je sploh možno izvesti.

Z vidika diplomske naloge je pomembno predvsem to, da iOS kot platforma ne omogoča resnične možnosti "popolnoma odprtokodnega telefona". Uporabnik niti teoretično nima možnosti zamenjati ključnih komponent sistema z odprtokodnimi alternativami. iOS je zato dober kontrast, saj pokaže, kako je videti maksimalno centraliziran mobilni ekosistem, kjer je nadzor nad napravo primarno v rokah proizvajalca [14].

2.3 Sistem Android

Android se pogosto predstavlja kot odprtokoden sistem, saj temelji na Androidovem odprtokodnem projektu (AOSP) [8]. To je pomembna razlika v primerjavi z iOS, ker projekt AOSP omogoča javno dostopno izvorno kodo za velik del platforme in teoretično dopušča razvoj alternativnih distribucij.

V praksi pa tipična naprava z Androidom ni enaka projektu AOSP. Večina naprav vsebuje:

- Prilagoditve izdelovalca originalne opreme (OEM), ki pogosto vključujejo

dodatne sistemske aplikacije;

- Zaprtokodne gonilnike in strojno programsko opremo za posamezne komponente, ki so del sklada dobavitelja [6, 9];

- Googlove mobilne storitve (GMS), ki niso del projekta AOSP in so na voljo le prek licence; pomemben del teh storitev predstavlja *GMS Core* oziroma *Google Play Services* [7, 3].

Posledica je, da je Android kot ekosistem hibrid. Odprtokodna osnova se v praksi nadgradi z več lastniškimi plastmi. To se najbolj pozna pri strojni podpori. Gonilniki za grafični pospeševalnik, kamero, mobilni modem in druge ključne komponente so pogosto zaprti ter vezani na konkretno jedro in različico sistema [6, 9]. Zaradi tega posodobitve niso zgolj vprašanje nove različice Androida, temveč tudi vprašanje združljivosti celotnega sklada gonilnikov in strojne programske opreme. Del problema Android poskuša nasloviti z modularizacijo sistemskih komponent, kjer je del posodobitev mogoče dostaviti ločeno od celotnega sistema [10].

Ena izmed posledic takšne arhitekture je tudi fragmentacija. Različni proizvajalci vzdržujejo svoje različice sistema in jedra, kar poveča stroške vzdrževanja in lahko negativno vpliva na hitrost ter trajanje zagotavljanja posodobitev [1, 36]. To je eden izmed razlogov, zakaj naprave pogosto dobijo le omejeno število večjih posodobitev, čeprav je strojna oprema še povsem zmogljiva. Kot minimalen referenčni okvir lahko uporabimo tudi zahteve programa za priporočene poslovne naprave, ki za vključene naprave predvideva podporo trenutni izdaji in vsaj eni večji nadgradnji operacijskega sistema [2].

2.4 Akterji mobilnega ekosistema

Za razumevanje omejitev odprtokodnih mobilnih sistemov je ključno upoštevati, da telefon ni samo operacijski sistem in aplikacije. Pri tipični napravi sodeluje več akterjev:

- **Proizvajalec operacijskega sistema oziroma ekosistema** (Apple ali Google, posredno tudi skupnost AOSP);
- **Izdelovalec originalne opreme (OEM)** (npr. OnePlus), ki prilagodi sistem;
- **Proizvajalec sistema na čipu (SoC)** (npr. Qualcomm), ki zagotovi gonilnike in strojno programsko opremo;
- **Operaterji** (v določenih državah tudi certificiranje in omejitve);
- **Razvijalci aplikacij**, ki se pogosto zanašajo na lastniške programske vmesnike (API) in storitve (npr. potisna obvestila, SafetyNet/Play Integrity) [5, 4].

Poseben primer je modemski podsistem. Ta je pogosto ločen procesor ali podsistem s svojo strojno programsko opremo, ki je praviloma lastniška [30, 18]. Zaradi varnosti in regulative je razumljivo, zakaj je ta del izoliran, vendar z vidika odprtokodnosti predstavlja strukturno omejitev, saj je strojna programska oprema praviloma lastniška in je uporabnik ne more realno zamenjati ali pregledati. Tudi če zamenjamo operacijski sistem in aplikacije, ostaja modemski podsistem ključen zaprt del naprave.

2.5 Alternativni Android ROM-i

Alternativni ROM-i za Android so pogost način, kako predvsem tehnično bolj podkovani uporabniki poskušajo povečati nadzor nad napravo. Najbolj poznan primer je LineageOS[1], poleg tega pa obstajajo še projekti, ki se posebej osredotočajo na zasebnost ali zmanjšanje odvisnosti od Googlovih storitev (npr. /e/OS[2], GrapheneOS[3]).

[1] `https://lineageos.org/`
[2] `https://e.foundation/e-os/`
[3] `https://grapheneos.org/`

Ena izmed pogostih razlik v primerjavi s tovarniškimi ROM-i je manj prednaložene programske opreme. Ker večina uporabnikov teh dodatkov ne uporablja, hkrati pa porabljajo prostor in sistemske vire, se je zanje uveljavil izraz *bloatware*, torej odvečna programska oprema.

Alternativni ROM-i pogosto ponujajo:

- manj prednaloženih sistemskih aplikacij;
- bolj transparentne nastavitve zasebnosti;
- več možnosti prilagajanja (npr. skrbniški dostop, drugačno obnovitveno okolje ter večji nadzor nad sistemskimi aplikacijami);
- v nekaterih primerih daljšo podporo za starejše naprave (odvisno od skupnosti in vzdrževanja) [17].

Ključna omejitev alternativnih ROM-ov je, da so še vedno vezani na isti problem kot tovarniški Android. Odvisni so od zaprtokodnih gonilnikov in strojne programske opreme, ki jih zagotavlja proizvajalec strojne opreme. Poleg tega se v zadnjih letih povečuje odvisnost aplikacij od lastniških storitev in mehanizmov preverjanja celovitosti (npr. Play Integrity), kar lahko na sistemih brez Googlovih storitev povzroča težave pri aplikacijah z višjimi varnostnimi zahtevami (npr. plačila, bančne aplikacije) [4, 25]. Pri zaščiti vsebin se uporabljajo mehanizmi upravljanja digitalnih pravic (DRM; angl. *Digital Rights Management*), kot je Widevine, kjer lahko razlike v varnostnih ravneh vplivajo na razpoložljive funkcionalnosti oziroma kakovost predvajanja [24, 59, 15, 16].

2.6 Mobilni Linux: postmarketOS, Ubuntu Touch in sorodni projekti

Mobilni Linux poskuša na telefon prinesti bolj klasičen model računalniškega sistema. V ospredju so standarden uporabniški prostor operacijskega sistema

GNU/Linux, paketni upravljalnik, terminal in večja svoboda prilagajanja. Med bolj aktivnimi projekti sta postmarketOS[4] in Ubuntu Touch[5].

Projekt postmarketOS gradi na distribuciji Alpine Linux in ima dolgoročno usmeritev v vzdržljivost ter ponovno uporabnost naprav [42]. S tem poskuša zmanjšati odvisnost od specifičnih različic Androida in cilja na to, da bi naprave lahko uporabljale jedro glavne razvojne veje, kjer je to izvedljivo.

Ubuntu Touch razvija skupnost UBports in poskuša ponuditi uporabniku prijazen mobilni sistem z lastnim grafičnim vmesnikom. Pomemben del zasnove je koncept konvergence, kjer lahko ista naprava deluje kot telefon in kot namizni računalnik [57, 56]. Cilj je ponuditi alternativo Androidu, ki ni vezana na Googlov ekosistem in kjer je večina programske opreme odprtokodna.

Kljub obetom je resničnost mobilnega Linuxa še vedno močno odvisna od strojne podpore. Pri številnih napravah se za delovanje še vedno uporabljajo obhodi, kot so vmesni sloji za uporabo Android gonilnikov (npr. libhybris, Halium), kar pomeni kompromis med odprtostjo in praktično uporabnostjo [29, 31, 49]. To je tipičen primer konflikta. Ideal predstavlja odprtokoden programski sklad, tipično v obliki GNU/Linux, vendar praksa pri mobilnih napravah pogosto zahteva vključitev obstoječih lastniških komponent.

2.7 Namenske Linux naprave

Poleg prenosov na obstoječe telefone Android obstajajo tudi namenske naprave, ki ciljajo na uporabnike, ki želijo odprtokodne alternative (npr. PinePhone, Librem 5) [40, 41, 44, 45]. Te naprave pogosto poskušajo izboljšati stanje tako, da izberejo strojno opremo z boljšo podporo v jedru Linuxa ali z bolj odprto dokumentacijo.

[4] `https://postmarketos.org/`
[5] `https://ubports.com/`

Prednost namenskih Linux telefonov je, da je programski sklad, ki sestoji iz jedra, uporabniškega prostora in uporabniškega vmesnika, pogosto bolj konsistenten in da skupnost razvoj usmeri na manjše število naprav. Slabost pa je, da se takšne naprave pogosto ne morejo kosati z najboljšimi telefoni s sistemom Android ali iOS glede zmogljivosti, kakovosti kamere in porabe energije. Zato ostajajo predvsem v niši, kar vpliva tudi na velikost skupnosti in hitrost razvoja.

2.8 Povzetek

Pregled področja pokaže, da mobilni ekosistem trenutno obvladuje duopol Android/iOS, kjer je uporabniška svoboda v veliki meri omejena z zaprtimi ekosistemi, lastniškimi komponentami in poslovnimi modeli. Android sicer ponuja odprtokodno osnovo (AOSP), vendar tipične naprave vsebujejo veliko zaprtih slojev, kar oteži dolgotrajno vzdrževanje in razvoj alternativ. Alternativni ROM-i za Android delno izboljšajo nadzor nad sistemom, vendar ostajajo odvisni od istih zaprtih gonilnikov in strojne programske opreme. Projekti mobilnega Linuxa ponujajo bolj radikalno alternativo, vendar je njihova praktična uporabnost še vedno močno omejena s strojno podporo in ekosistemom aplikacij. To neposredno motivira nadaljnja poglavja, kjer bodo najprej opredeljeni kriteriji "popolnoma odprtokodnega telefona", nato pa bodo analizirane obstoječe rešitve in empirično preizkušene.

Poglavje 3

Teoretična izhodišča

V prejšnjem poglavju smo pokazali, da je tudi pri sistemih z odprtokodno osnovo velik del mobilnega ekosistema odvisen od zaprtih komponent. Da lahko kasneje ocenimo, v kolikšni meri je "popolnoma odprtokoden telefon" v praksi izvedljiv, moramo najprej definirati osnovne koncepte, sloje sistema in tipične omejitve. V tem poglavju opredelimo razliko med odprtokodno in prosto programsko opremo, na kratko predstavimo arhitekturo mobilnih sistemov ter izpostavimo vlogo gonilnikov in strojne programske opreme. Nato povežemo odprtost z varnostjo in zasebnostjo, pri čemer obravnavamo tudi tveganja skritih funkcionalnosti oziroma stranskih vrat (v literaturi pogosto označenih kot angl. *backdoor*).

3.1 Odprtokodna in prosta programska oprema

Pojma odprtokodna programska oprema (angl. *open source software*) in prosta programska oprema (angl. *free software*) se pogosto uporabljata kot sopomenki, vendar izhajata iz različnih poudarkov. Open Source Initiative (OSI) odprtokodnost definira s kriteriji, ki med drugim zahtevajo dostop do izvorne kode, možnost sprememb in redistribucije ter nediskriminatorne licence [39]. Free Software Foundation (FSF) pa v ospredje postavlja uporabniške svoboščine: program je prost, če omogoča (1) poganjanje za polju-

ben namen, (2) preučevanje delovanja, (3) prilagajanje ter (4) deljenje kopij, tudi spremenjenih [22]. V nadaljevanju zato uporabljamo tudi širši izraz prosta in odprtokodna programska oprema (FOSS; angl. *Free and Open Source Software*), kadar želimo zajeti oba vidika.

Pri ocenjevanju "odprtosti telefona" je pomembno, da ločimo licenco od samega dejstva, da je koda javno dostopna. V praksi se pojavljajo tudi projekti, kjer je izvorna koda objavljena, vendar licenca ne dopušča redistribucije ali sprememb; za tak model se pogosto uporablja izraz dostopna izvorna koda (angl. *source-available*). Za kriterije v tej diplomski nalogi zato ne zadošča zgolj javna objava kode, ampak zahtevamo tudi pravice, ki jih določajo licence, ter možnost dejanske uporabe spremenjenih različic (npr. namestitev na napravo).

3.1.1 Licence in posledice za mobilne sisteme

Licence določajo pogoje uporabe, spreminjanja in redistribucije izvorne kode ter izpeljanih del. V grobem jih lahko razdelimo na:

- **permisivne licence** (npr. MIT [54], BSD [52], Apache 2.0 [12]), ki praviloma omogočajo vključitev kode tudi v lastniške izdelke, ter

- **licence z zahtevami deljenja izboljšav** (angl. *copyleft*, npr. GPLv2 [23]), ki ob redistribuciji pod določenimi pogoji zahtevajo, da so izpeljana dela na voljo pod enako ali združljivo licenco.

Za mobilne naprave je posebej pomembno jedro Linux, ki je licencirano pod GPLv2 [23]. To proizvajalcem pri distribuciji naprav nalaga obveznosti glede objave izvorne kode sprememb jedra, vendar še ne pomeni, da je odprt celoten programski sklad. V praksi lahko ostanejo zaprti posamezni gonilniki, strojna programska oprema ter velik del uporabniškega prostora (npr. sistemske aplikacije ali dobaviteljske knjižnice).

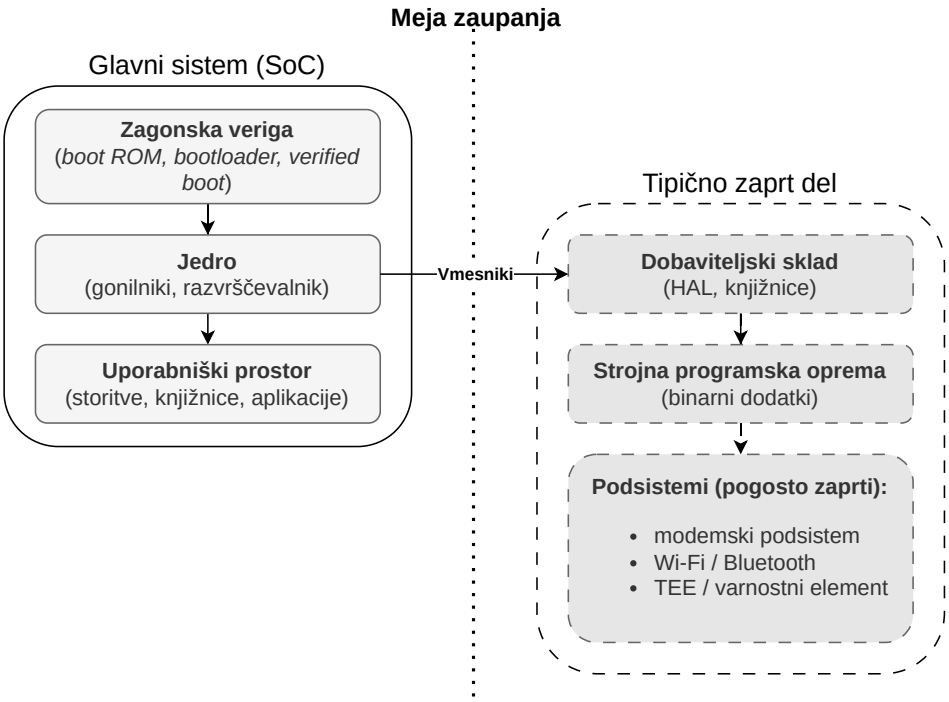
3.2 Arhitektura sistema: od zagona do uporabniškega prostora

Pametni telefon je sestavljen iz več podsistemov. Poleg glavnega procesorja (SoC) ima naprava pogosto ločene mikrokrmilnike ali koprocesorje za modem, Wi-Fi/Bluetooth, kamero in senzorje, pogosto pa tudi varnostne podsisteme, kot je zaupanja vredno izvajalno okolje (TEE; angl. *Trusted Execution Environment*). Ti deli praviloma uporabljajo lastno strojno programsko opremo in so pomemben del vprašanja zaupanja, ker so za uporabnika običajno netransparentni.

Na nivoju glavnega sistema ločimo:

- zagonsko verigo (angl. *bootchain*), ki določa, kaj se sploh sme zagnati (npr. zagonski nalagalnik) in preverjen zagon (angl. *verified boot*),
- jedro z gonilniki in
- uporabniški prostor, kjer tečejo sistemske storitve, knjižnice in aplikacije.

Slika 3.1 povzema sloje sistema in tipične zaprte komponente, ki predstavljajo glavno omejitev pri popolnoma odprtokodnem telefonu.



Slika 3.1: Poenostavljena arhitektura telefona z vidika slojev sistema in tipičnih zaprtih komponent.

3.2.1 Linux: glavna razvojna veja jedra in vzdrževanje

Sklad GNU/Linux je družina sistemov, ki delijo isto jedro, nad njim pa lahko teče različna programska oprema uporabniškega prostora. Za dolgoročno vzdrževanje naprav je ključno, ali je podpora za strojno opremo vključena v glavno razvojno vejo jedra ali pa je vezana na proizvajalčevo prilagojeno različico jedra [53]. Če je podpora del glavne veje, lahko naprava praviloma dobiva varnostne popravke skupaj z ostalim ekosistemom. Če pa je odvisna od zaprtih ali slabo vzdrževanih sprememb dobavitelja, je življenjska doba programske podpore praviloma krajša.

3.2.2 Android: AOSP in ločitev dobaviteljskega sklada

Android temelji na jedru Linux, vendar ima specifičen uporabniški prostor in nabor sistemskih storitev, ki skupaj tvorijo platformo [8]. Za odprtost je pomembna meja med generičnim sistemom in strojno specifičnimi deli. Projekt Treble je uvedel bolj jasno ločitev na particije in vmesnike (npr. ločen `system` in `vendor`), s čimer sistem teoretično omogoča posodabljanje neodvisno od dobaviteljskega sklada [6, 9]. Kljub temu dobaviteljski del pogosto vsebuje zaprt nabor knjižnic, implementacije abstrakcijskega sloja strojne opreme (HAL), gonilnike in strojno programsko opremo, brez katerih naprava ne deluje.

3.3 Gonilniki, strojna programska oprema in binarni dodatki

Pri telefonih se pogosto srečamo s situacijo, kjer je del kode odprtokoden, ključne funkcionalnosti pa so dobavljene v obliki binarnih dodatkov. Pomembno je ločiti med:

Gonilniki kodo, ki teče v jedru ali uporabniškem prostoru in omogoča komunikacijo s strojno opremo.

Strojno programsko opremo kodo, ki teče na sami komponenti (npr. Wi-Fi čip, digitalni signalni procesor ali modem) in jo sistem zgolj naloži ali upravlja.

Tudi če so gonilniki odprtokodni, je strojna programska oprema lahko popolnoma zaprta, brez možnosti revizije ali popravkov. To je ena glavnih tehničnih ovir za telefon, kjer bi bil celoten programski sklad pregledljiv in zamenljiv. Najbolj problematični so podsistemi z visokimi privilegiji ali neposrednim vplivom na komunikacijo (npr. modemski podsistem in brezžični vmesniki). Raziskave kažejo, da se ranljivosti in napadi pogosto selijo prav v zaprt dobaviteljski sklad in strojno programsko opremo, kjer je preglednost manjša in so orodja za analizo omejena [30, 18].

3.4 Odprtost, varnost in zasebnost

Odprta koda lahko prispeva k varnosti, ker omogoča neodvisen pregled in hitrejše odkrivanje napak. Vendar odprtost sama po sebi ni garancija; varnost je predvsem rezultat procesa (revizije, testiranja, odzivnost na ranljivosti) in nadzora nad tem, kaj se na napravi dejansko izvaja. Pri telefonih zato poleg licence ocenjujemo tudi oskrbovalno verigo, mehanizme posodabljanja ter stopnjo zaprtih komponent.

3.4.1 Stranska vrata in oskrbovalna veriga

Stranska vrata (angl. *backdoor*) razumemo kot namerno dodano ali prikrito funkcionalnost, ki omogoča dostop mimo običajnih varnostnih mehanizmov. Pri zaprtih komponentah je problem očiten: uporabnik in neodvisni raziskovalci težko preverijo, ali takšna funkcionalnost obstaja. Tudi pri odprtokodni programski opremi pa ostaja pomemben napadni vektor oskrbovalna veriga in orodja za izgradnjo. Klasičen primer je Thompsonov opis, kako lahko kompromitiran prevajalnik vstavi stranska vrata v program tudi takrat, ko je izvorna koda navidezno "čista" [55]. Za oceno zaupanja je zato smiselno gledati širše od same izvorne kode.

Pri telefonih se pomisleki o stranskih vratih najpogosteje vežejo na:

- zaprt dobaviteljski sklad na komunikacijskih čipih (modem, Wi-Fi/Bluetooth),
- privilegirane varnostne podsisteme (npr. TEE in varnostni element) in
- lastniške sistemske storitve s širokimi dovoljenji (npr. telemetrija).

3.4.2 Celovitost zagona in ponovljive gradnje

Moderni telefoni pogosto uporabljajo mehanizme preverjenega zagona, kjer je zagonska veriga kriptografsko vezana na podpisane sistemske slike. To izboljša zaščito pred zlonamernimi spremembami, hkrati pa lahko omeji lastnika naprave, če je zagonski nalagalnik zaklenjen in proizvajalec ne omogoča nalaganja alternativnih podpisov [11]. Za sisteme, ki ciljajo na odprtost, je zato ključno ravnotežje: želimo visoko stopnjo celovitosti zagonske verige, hkrati pa možnost, da lastnik na napravo naloži in s svojimi ključi podpiše lastno različico sistema.

Dodaten sloj zaupanja predstavljajo ponovljive gradnje (angl. *reproducible builds*), kjer lahko neodvisna stran iz iste izvorne kode izdela identične binarne pakete [48]. S tem zmanjšamo tveganje, da bi bila uradna izdaja kompromitirana na poti od kode do binarne slike. Pri telefonih je to posebej relevantno, ker uporabniki praviloma nameščajo vnaprej zgrajene sistemske slike, ne pa da bi sistem prevajali sami.

3.4.3 Zasebnost kot posledica arhitekture

Zasebnost pri telefonih ni samo vprašanje aplikacij, ampak tudi sistemske arhitekture. Če naprava temelji na zaprtih storitvah in naborih API-jev, uporabnik težko oceni, kateri podatki se pošiljajo in kdaj. Alternativni sistemi lahko zmanjšajo odvisnost od centraliziranih storitev, vendar se pri tem pogosto pojavi kompromis pri združljivosti aplikacij, zlasti pri aplikacijah z višjimi varnostnimi zahtevami, ki uporabljajo mehanizme preverjanja celovitosti [4].

3.5 Vzdrževanje in življenjski cikel naprav

Pri oceni odprtokodnega telefona je praktična uporabnost močno povezana z vzdrževanjem. Če naprava ne dobiva varnostnih popravkov, odprtost hitro postane zgolj formalna lastnost. Analize ekosistema Android kažejo velike razlike v dolžini podpore in zamudah pri varnostnih posodobitvah med proizvajalci [1]. V mobilnem Linuxu se ta problem pogosto rešuje z uporabo glavne razvojne veje jedra in standardnega uporabniškega prostora, vendar je cena praviloma slabša strojna podpora in manj zrel ekosistem aplikacij.

V naslednjih poglavjih bomo na podlagi teh izhodišč postavili merljive kriterije, kaj v našem kontekstu pomeni "popolnoma odprtokoden telefon", nato pa bomo z njimi analizirali izbrane obstoječe rešitve.

Poglavje 4

Model "popolnoma odprtokodnega telefona"

V prejšnjih poglavjih smo pokazali, da mobilni telefon ni le operacijski sistem z aplikacijami, temveč preplet več slojev programske opreme, strojne opreme, dobaviteljskih komponent in mehanizmov zagona. Zato pojem "popolnoma odprtokoden telefon" v praksi ni samoumeven: pri pametnem telefonu odprtost ni odvisna le od licence operacijskega sistema, temveč tudi od gonilnikov, strojne programske opreme, sistemskih storitev, načina nameščanja aplikacij in možnosti dolgoročnega vzdrževanja.

V tem poglavju najprej opredelimo idealni referenčni model popolnoma odprtokodnega telefona. Nato določimo kriterije, s katerimi bomo konkretne rešitve primerjali glede na dve ločeni dimenziji: **odprtost** in **praktično uporabnost**. Odprtost sama po sebi še ne pomeni, da je naprava uporabna kot vsakdanji telefon, visoka uporabnost pa še ne pomeni, da rešitev uporabniku omogoča dejanski nadzor nad sistemom. Zato moramo obe dimenziji obravnavati ločeno.

4.1 Idealni referenčni model

Če pojem "popolnoma odprtokoden" razumemo dobesedno, bi to pomenilo, da je vsa programska oprema, ki se izvaja na napravi, na voljo kot prosta in odprtokodna programska oprema, z možnostjo pregleda, sprememb in redistribucije [39, 22]. To ne vključuje le uporabniškega prostora in jedra, temveč tudi gonilnike, zagonsko verigo, sistemske storitve ter strojno programsko opremo pomožnih podsistemov.

Tak idealni telefon bi moral hkrati izpolnjevati več pogojev. Celoten sistem bi moral biti zgradljiv iz izvorne kode, ključne komponente pa bi morale biti neodvisno preverljive, po možnosti tudi s ponovljivimi gradnjami [48]. Lastnik naprave bi moral imeti možnost na dokumentiran in ponovljiv način namestiti lasten sistem, ne da bi se moral pri tem odpovedati preverjenemu zagonu ali postati popolnoma odvisen od proizvajalca. Pomembne funkcije telefona bi morale ostati uporabne tudi brez zaprtih, centraliziranih ali proizvajalčevo vezanih storitev.

V kontekstu sodobnih pametnih telefonov je tak ideal praviloma težko dosegljiv. V praksi se skoraj vedno pojavijo vsaj tri pomembne omejitve: (1) zaprt modemski podsistem, (2) zaprta strojna programska oprema za Wi-Fi/Bluetooth, kamero, grafični procesor in druge podsisteme, ter (3) mehanizmi preverjenega zagona, ki lahko omejijo nalaganje lastnih gradenj, če lastnik nima nadzora nad ključi oziroma je zagonski nalagalnik zaklenjen [11].

V nadaljevanju zato ta idealni telefon obravnavamo kot referenčno točko. Obstoječih rešitev ne ocenjujemo binarno, temveč glede na to, kako blizu so temu idealu v dveh dimenzijah: (1) **odprtost** in (2) **praktična uporabnost**.

4.2 Slojni pogled na odprtost telefona

Za ocenjevanje odprtosti potrebujemo slojni model, ki je skladen z arhitekturo iz slike 3.1. V nadaljevanju uporabljamo naslednje sloje:

1. **Zagonska veriga**: od začetnega zagona do nalaganja jedra in upo-

rabniškega prostora.

2. **Jedro in gonilniki**: jedro Linux ter gonilniki v jedru oziroma uporabniškem prostoru.

3. **Strojna programska oprema**: programska oprema pomožnih podsistemov (modem, Wi-Fi/Bluetooth, grafični procesor, kamera, senzorji ipd.).

4. **Sistemske storitve in vmesni sloj** (angl. *middleware*): npr. sistemske storitve in abstrakcijska plast strojne opreme v Androidu oziroma klasične storitve GNU/Linux v uporabniškem prostoru.

5. **Privzete aplikacije in distribucija aplikacij**: osnovni nabor aplikacij ter način nameščanja in posodabljanja dodatnih aplikacij.

Tak slojni pogled je primeren za ocenjevanje odprtosti, ker omogoča, da ločeno obravnavamo različne vrste zaprtosti. Rešitev je lahko na primer razmeroma odprta na ravni uporabniškega prostora, a močno omejena na ravni strojne programske opreme. Prav zato odprtosti ne obravnavamo kot ene same lastnosti, temveč kot profil po posameznih slojih sistema.

4.3 Kriteriji odprtosti po slojih

V tem razdelku postavimo kriterije, s katerimi bomo kasneje primerjali konkretne rešitve. Kriteriji so oblikovani tako, da so čim bolj preverljivi: zanima nas, ali je nekaj javno objavljeno, neodvisno zgradljivo, dokumentirano, dolgoročno vzdrževano ali zamenljivo.

4.3.1 Zagonska veriga

Pri zagonski verigi nas zanima predvsem, ali ima lastnik naprave realno možnost namestiti lasten sistem, ne da bi bil pri tem popolnoma odvisen od proizvajalca. Pri tem ni pomembna le možnost odklenitve naprave, temveč

tudi vprašanje, ali je mogoče ohraniti koncept preverjenega zagona in razumne varnostne lastnosti sistema.

Kriteriji:

- **Odklep zagonskega nalagalnika**: ali je uradno podprt in dokumentiran ter ali ne zahteva nepovratnih korakov, ki uporabnika trajno omejijo.

- **Preverjen zagon z uporabniškimi ključi**: ali sistem omogoča preverjen zagon, pri čemer lahko naprava zaupa tudi podpisom, ki jih upravlja lastnik [11].

- **Dokumentiranost namestitve**: ali je postopek namestitve alternativnega sistema javno opisan in reproducibilen.

4.3.2 Jedro in gonilniki

Pri jedru in gonilnikih je ključno vprašanje dolgoročno vzdrževanje. Če je podpora za napravo vključena v glavno razvojno vejo jedra Linux, je možnost dolgotrajnih varnostnih posodobitev praviloma večja [53]. Nasprotno pa dobaviteljske izpeljanke in zaprtokodni gonilniki povečujejo odvisnost od proizvajalca ter otežujejo neodvisno vzdrževanje.

Kriteriji:

- **Podpora v glavni veji**: ali so ključni gonilniki vključeni v uradno jedro ali pa gre za dobaviteljsko izpeljanko.

- **Odprtost gonilnikov**: ali so gonilniki odprtokodni ali pa so prisotni binarni dodatki v jedru oziroma uporabniškem prostoru.

- **Vzdrževalni signal**: ali je vidna aktivnost posodobitev, varnostnih popravkov in vključevanja sprememb v izvorni projekt.

4.3.3 Strojna programska oprema

Strojna programska oprema je praviloma največja ovira za strogo odprt telefon. Pri sodobnih pametnih telefonih je velik del te programske opreme tesno vezan na dobavitelje čipov, strojno zasnovo naprave in regulativne zahteve. Popolna odprtost zato pogosto ni realno dosegljiva.

V takem primeru je pomembno tudi vprašanje, kako močno je zaprt podsistem vpet v preostali sistem. Pri ocenjevanju zato ne upoštevamo le tega, ali je posamezna komponenta odprta, temveč tudi, ali je njen vpliv mogoče omejiti z arhitekturo, ločenimi privilegiji in izolacijo. Cilj ni "narediti zaprto komponento odprto", temveč zmanjšati njen vpliv na varnost, zasebnost in dolgoročno vzdrževanje naprave.

Kriteriji:

- **Razpoložljivost izvorne kode**: ali je strojna programska oprema odprtokodna; če ni, ali obstajajo odprte alternative, četudi z omejitvami.
- **Obseg in privilegiji**: kateri podsistemi so zaprti in kako privilegirani so; modemski podsistem je praviloma pomembnejši od strojnega programa posameznega senzorja.
- **Izolacija**: ali arhitektura zmanjšuje vpliv zaprtih komponent, saj ranljivosti v zaprtih skladih predstavljajo realno napadalno površino [30, 18].

4.3.4 Sistemske storitve in vmesni sloj

Pri Androidu je pomembna meja med odprtim jedrom platforme (AOSP) ter dobaviteljskimi komponentami in implementacijami abstrakcijske plasti strojne opreme, pri mobilnem GNU/Linux pa predvsem to, ali je uporabniški prostor sestavljen iz standardnih paketov distribucije oziroma iz posebnega, projektno vezanega sklada.

Kriteriji:

- **Delež komponent FOSS**: koliko ključnih sistemskih storitev je odprtokodnih in zamenljivih.
- **Odvisnost od lastniških storitev**: ali sistem za osnovno delovanje zahteva centralizirane, zaprtokodne storitve ali pa ima odprte alternative.
- **Ponovljive gradnje**: ali je realno mogoče neodvisno preveriti binarne izdaje, vsaj za kritične dele sistema [48].

4.3.5 Privzete aplikacije in distribucija aplikacij

Na zadnjem sloju se pokaže praktična plat sistema: tudi zelo odprt sistem je za mnoge uporabnike težko uporaben, če nima varnega in preglednega načina nameščanja aplikacij. V tej nalogi kot ustrezen način razumemo takšnega, ki omogoča nameščanje in posodabljanje aplikacij iz zaupanja vrednega vira brez nestandardnih postopkov ter s preverljivimi posodobitvami.

Kriteriji:

- **Privzeto nameščene aplikacije**: ali so aplikacije iz privzete namestitve odprtokodne ter ali imajo minimalno, dokumentirano in uporabniško nadzorovano telemetrijo.
- **Trgovina in repozitoriji**: ali je primarni kanal namestitve aplikacij odprt (npr. paketni repozitorij ali trgovina z aplikacijami) ter ali podpira preverljive posodobitve.
- **Neodvisnost od proizvajalca**: ali uporabnik lahko vzdržuje sistem in aplikacije tudi po tem, ko proizvajalec naprave uradno zaključi podporo.

4.3.6 Povzetek

Za lažji pregled tabela 4.1 povzema sloje in tipične kriterije odprtosti, ki jih bomo uporabili v nadaljevanju.

Sloj	Kaj preverjamo
Zagonska veriga	odklep zagonskega nalagalnika; preverjen zagon z uporabniškimi ključi; dokumentiran postopek namestitve
Jedro in gonilniki	podpora v glavni veji; odprtost gonilnikov; aktivno vzdrževanje
Strojna programska oprema	odprta koda ali odprte alternative; obseg zaprtih podsistemov; izolacija
Sistemske storitve	delež komponent FOSS; odvisnost od lastniških storitev; ponovljive gradnje
Aplikacije in distribucija	odprtokodne privzete aplikacije; odprti repozitoriji; neodvisnost od proizvajalca

Tabela 4.1: Povzetek slojev in tipičnih kriterijev odprtosti v predlaganem modelu.

4.4 Kriteriji praktične uporabnosti

Ker je ciljna naprava telefon, mora sistem pokriti vsaj osnovne funkcionalnosti, sicer primerjava ni smiselna. Odprt sistem brez delujočih klicev je lahko zanimiv kot tehnična demonstracija, vendar ne deluje kot telefon v običajnem pomenu besede.

Praktične uporabnosti v tej nalogi ne ocenjujemo po slojih sistema, temveč po funkcionalnih sklopih, ki skupaj določajo, ali je naprava primerna za vsakodnevno rabo. Izhodišče je minimalni nabor funkcionalnosti za vsakodnevno rabo: naprava mora omogočati osnovno telefonijo, povezljivost, dostop do ključnih aplikacij ter razumno enostavno namestitev in vzdrževanje.

V nadaljevanju uporabljamo naslednje sklope:

- **Osnovne funkcije telefona**: klici, SMS, mobilni podatki in stabilnost povezave.
- **Ključna strojna podpora**: kamera, GPS, Bluetooth, Wi-Fi, zvok in senzorji; po potrebi tudi NFC.

- **Avtonomija in zmogljivost**: poraba energije v mirovanju in pri rabi, segrevanje ter odzivnost sistema.
- **Ekosistem aplikacij**: ali so za minimalni nabor vsakodnevnih nalog na voljo ustrezne aplikacije ali spletne alternative.
- **Posodobitve in vzdrževanje**: kako pogosto prihajajo varnostne posodobitve in kako enostavno jih je namestiti.

Pri teh sklopih nas zanima predvsem, ali posamezne funkcije delujejo dovolj zanesljivo, da je naprava primerna za vsakodnevno rabo.

Pri ekosistemu aplikacij v tej nalogi preverjamo, ali so za vnaprej opredeljen minimalni nabor vsakodnevnih nalog na voljo ustrezne aplikacije ali spletne alternative. Kot minimalne kategorije upoštevamo: komunikacijo, navigacijo, spletni brskalnik, e-pošto, kamero in galerijo, predvajanje medijev ter osnovne aplikacije za avtentikacijo, npr. dvofaktorsko avtentikacijo. Posebej obravnavamo tudi aplikacije z višjimi varnostnimi zahtevami, kot so bančne aplikacije in storitve mobilnega plačevanja, saj pogosto uporabljajo mehanizme preverjanja celovitosti. Takšne aplikacije so zato dober pokazatelj praktičnih omejitev posamezne rešitve [4].

4.5 Način ocenjevanja

Za kasnejšo analizo projektov in študijo primera potrebujemo konsistenten način primerjave. Posamezne rešitve zato ocenjujemo v dveh ločenih dimenzijah: (1) **odprtost** in (2) **praktična uporabnost**.

Pomembno je, da pri tem ne ocenjujemo projekta povsem abstraktno, temveč **konkretno rešitev na konkretni podprti napravi**. Na primer: ne ocenjujemo "GrapheneOS na splošno", temveč "GrapheneOS na uradno podprti napravi". Na ta način se izognemo mešanju lastnosti programske rešitve z lastnostmi strojne opreme.

Odprtost ocenjujemo po slojih iz sekcije 4.3, praktično uporabnost pa po sklopih iz sekcije 4.4. V obeh dimenzijah uporabimo tristopenjsko lestvico z

vrednostmi 0, 1 in 2.

Lestvica odprtosti ($O \in \{0, 1, 2\}$).

- 0 – "pretežno zaprto": ključni deli so lastniški, brez realnih odprtih alternativ ali brez možnosti neodvisnega vzdrževanja.
- 1 – "mešano": prisotna je kombinacija odprtih in zaprtih komponent, pri čemer odprti del ne pokrije celotne funkcionalnosti ali zahteva pomembne kompromise.
- 2 – "pretežno odprto": večina funkcionalnosti je pokrita z odprto programsko opremo in omogoča razumno stopnjo neodvisnega vzdrževanja.

Lestvica uporabnosti ($U \in \{0, 1, 2\}$).

- 0 – "neuporabno": ključna funkcionalnost ne deluje ali pa je delovanje preveč nezanesljivo za vsakodnevno rabo.
- 1 – "omejeno": funkcionalnost deluje, vendar z opaznimi omejitvami, pogostimi težavami ali zmanjšano zanesljivostjo.
- 2 – "uporabno": funkcionalnost deluje stabilno in brez posebnih omejitev za tipične scenarije uporabe.

Ocena posameznega sloja oziroma sklopa je sintezna ocena na podlagi vnaprej določenih kriterijev. Pri tem ne računamo uteženega skupnega rezultata, saj bi tak pristop ustvaril vtis večje objektivnosti, kot jo je pri tovrstni primerjavi realno mogoče zagotoviti. Namesto tega rezultate predstavimo v obliki tabel in radarskih diagramov, ločeno za odprtost in za praktično uporabnost.

Tak način prikaza omogoča, da pri vsaki rešitvi jasno pokažemo njen profil kompromisov, torej kje je bližje idealu popolnoma odprtokodnega telefona in kje od njega bistveno odstopa. V razpravi lahko te profile dodatno interpretiramo tudi z vidika različnih tipov uporabnikov, vendar to ne spremeni osnovnega načina ocenjevanja.

Pri analizi obstoječih rešitev bodo ocene temeljile na javno dostopni dokumentaciji projektov, seznamih podprtih funkcionalnosti, znanih omejitvah ter uporabniških poročilih. V empiričnem delu pa bomo iste sklope ocenili še na podlagi lastnega testiranja na izbrani napravi. Ocene v primerjalni analizi so zato nujno bolj okvirne kot ocene v študiji primera, kjer lahko funkcionalnosti preverimo neposredno.

4.6 Povzetek

V tem poglavju smo idealni popolnoma odprtokodni telefon opredelili kot referenčno točko za nadaljnjo primerjavo. Nato smo določili kriterije odprtosti po slojih sistema in kriterije praktične uporabnosti po funkcionalnih sklopih telefona. Posamezne rešitve bomo v nadaljevanju ocenjevali ločeno v obeh dimenzijah, brez združevanja v en sam skupni rezultat.

Tak pristop omogoča bolj pregledno in pošteno primerjavo. Namesto navidezno natančne končne številke dobimo profil posamezne rešitve, iz katerega je razvidno, kje so njene prednosti, kje omejitve in kateri kompromisi so posledica konkretnih tehničnih ali organizacijskih odločitev projekta.

Poglavje 5

Analiza trenutnega stanja odprtokodnih mobilnih rešitev

V prejšnjem poglavju smo definirali model, s katerim lahko posamezne mobilne rešitve primerjamo glede na dve ločeni dimenziji: **odprtost** in **praktična uporabnost**. V tem poglavju ta model uporabimo za analizo obstoječih odprtokodnih mobilnih rešitev.

Namen poglavja ni zgolj izdelati seznama projektov, temveč pokazati, kako posamezne rešitve v praksi različno razporejajo kompromise med odprtostjo, varnostjo, zrelostjo programske podpore in vsakdanjo uporabnostjo. Pri tem nas ne zanima le, ali je projekt "odprt", temveč tudi, *kateri del* sistema je odprt, kje ostajajo odvisnosti od zaprtih komponent in kako močno to vpliva na dejansko uporabo naprave.

Analizirane rešitve razdelimo v tri večje skupine: (1) alternativne ROM-e za sistem Android, (2) mobilne distribucije sistema GNU/Linux na obstoječi strojni opremi za sistem Android in (3) namenske telefone z operacijskim sistemom GNU/Linux. Takšna delitev je smiselna, ker posamezne skupine rešujejo isti problem z različnimi tehničnimi izhodišči. Alternativni ROM-i za sistem Android praviloma ohranijo največ praktične uporabnosti, mobilne distribucije sistema GNU/Linux pogosto povečajo odprtost uporabniškega prostora, namenski telefoni z operacijskim sistemom GNU/Linux pa so naj-

bližje idealu arhitekturnega nadzora nad celotnim sistemom.

V nadaljevanju za celoten operacijski sistem uporabljamo izraz GNU/Linux, medtem ko izraz Linux uporabljamo le, kadar govorimo o jedru Linux.

V tem poglavju uporabljamo kriterije odprtosti iz sekcije 4.3 in kriterije praktične uporabnosti iz sekcije 4.4. Ker gre predvsem za sintezo javno dostopne dokumentacije projektov, seznamov podprtih naprav, znanih omejitev in razvojnih usmeritev, so ocene v tem poglavju nujno bolj okvirne kot v empiričnem delu naloge. Kljub temu takšna analiza omogoča dovolj jasen pregled nad tem, katere vrste kompromisov so značilne za posamezno družino rešitev.

5.1 Alternativni ROM-i za sistem Android

Alternativni ROM-i za sistem Android so danes najpomembnejša skupina odprtokodnih mobilnih rešitev za uporabnike, ki jih zanima razmeroma visoka stopnja praktične uporabnosti ob hkratnem zmanjšanju odvisnosti od tovarniškega sistema. Njihova ključna prednost je, da praviloma ohranijo združljivost z aplikacijskim ekosistemom sistema Android in s tem velik del vsakdanje uporabnosti telefona.

Hkrati pa gre tudi za skupino rešitev, pri kateri hitro postane jasno, da odprtost mobilnega operacijskega sistema ni enaka odprtosti celotne naprave. Čeprav ti projekti večinoma temeljijo na odprtokodnem jedru platforme Android, pri konkretni podpori posamezni napravi praviloma ostanejo vezani na dobaviteljske komponente, zaprtokodne implementacije abstrakcijske plasti strojne opreme, binarne dodatke in strojno programsko opremo posameznih podsistemov [8, 6, 9]. Zato so alternativni ROM-i za sistem Android praviloma močnejši na ravni uporabniškega prostora in sistemskih storitev kot pa na ravni gonilnikov, strojne programske opreme in celotne zagonske verige.

V tej skupini obravnavamo štiri reprezentativne primere: LineageOS[1],

[1] `https://lineageos.org/`

GrapheneOS[2], /e/OS[3] in Replicant[4]. Izbrani so zato, ker predstavljajo štiri precej različne strategije: široko podprt splošnonamenski ROM, varnostno usmerjen ROM za ožji nabor naprav, zasebnosti prijazno in uporabniško usmerjeno alternativo brez Googlovih storitev ter strogo svobodno rešitev za sistem Android, ki zavrača običajne pragmatične kompromise.

5.1.1 LineageOS

Projekt LineageOS je eden najbolj razširjenih predstavnikov alternativnih ROM-ov za sistem Android. Predstavlja se kot prost in odprtokoden operacijski sistem za različne naprave, ki temelji na platformi Android, pri čemer za uradno podprte naprave objavlja tudi natančna navodila za namestitev in gradnjo sistema [35, 32].

Z vidika modela iz prejšnjega poglavja je glavna prednost projekta LineageOS v tem, da razmeroma uspešno združi odprtokoden sistemski sloj z dovolj visoko stopnjo praktične uporabnosti. Uporabnik dobi sistem brez tovarniške preobleke proizvajalca, brez obvezne vezanosti na Googlove privzete aplikacije in z razmeroma preglednim postopkom namestitve ter posodabljanja [34]. Za številne naprave zato LineageOS predstavlja eno najbolj realističnih možnosti, kako podaljšati življenjsko dobo telefona in hkrati pridobiti več nadzora nad sistemom [17].

Vendar je LineageOS hkrati tudi dober primer omejitev, ki jih ima skoraj vsak ROM za sistem Android. Uradni odgovori na pogosta vprašanja poudarjajo, da projekt za vzdrževane naprave redno vključuje varnostne popravke iz projekta AOSP, medtem ko so varnostni popravki za dobaviteljski del sistema še vedno vezani na izvor zaprtih komponent iz tovarniškega sistema [33]. To pomeni, da LineageOS sicer poveča odprtost zgornjih slojev sistema, ne odpravi pa odvisnosti od proizvajalčevih binarnih komponent v spodnjem delu sklada.

[2] `https://grapheneos.org/`
[3] `https://e.foundation/e-os/`
[4] `https://replicant.us/`

Po našem modelu bi zato LineageOS praviloma ocenili kot rešitev z mešanim do pretežno odprtim profilom na ravni sistemskih storitev, privzetih aplikacij in načina distribucije aplikacij, bistveno slabše pa na ravni strojne programske opreme ter dela gonilnikov. Na ravni praktične uporabnosti je njegov profil praviloma močan: osnovne funkcije telefona, aplikacijski ekosistem in splošna odzivnost sistema so pri uradno podprtih napravah praviloma dovolj dobri za vsakodnevno rabo. Glavne omejitve se pokažejo pri funkcijah, ki so posebej odvisne od proizvajalčeve implementacije, ter pri aplikacijah, ki preverjajo celovitost naprave ali se opirajo na dodatne zaprtokodne storitve [4].

LineageOS je zato najprimernejši predstavnik "uravnoteženega kompromisa": ne približa se najbolj idealu popolnoma odprtokodnega telefona, je pa pogosto ena najbolj uporabnih odprtokodnih alternativ v praksi.

5.1.2 GrapheneOS

Projekt GrapheneOS predstavlja drugačno smer razvoja kot LineageOS. Ne osredotoča se na široko podporo številnim napravam, temveč na zasebnost, varnost in tehnično strogost pri ožjem naboru uradno podprtih telefonov Pixel [28, 27]. Prav ta omejenost ni naključna, temveč je del osnovne zasnove projekta.

Za naš model je GrapheneOS zanimiv predvsem zato, ker pokaže, da večja varnost in večji nadzor nad sistemom ne pomenita nujno tudi večje odprtosti vseh slojev. Na ravni zagonske verige je projekt zelo močan: postopek namestitve je dobro dokumentiran, vezan na uradno podprte naprave, po namestitvi pa predvideva ponovno zaklepanje zagonskega nalagalnika, tako da se ohrani preverjen zagon in zaščita pred nalaganjem nepooblaščenih sprememb sistema [26]. To je pomembna razlika v primerjavi s številnimi drugimi alternativnimi ROM-i, pri katerih uporabnik po namestitvi pogosto ostane pri trajno odklenjenem zagonskem nalagalniku.

Na drugi strani projekt GrapheneOS ni bistveno bolj odprt v spodnjih slojih strojne podpore. Še vedno je vezan na Googlovo strojno osnovo tele-

fonov Pixel, na dobaviteljske komponente strojne programske opreme in na gonilnike, ki jih zagotavlja proizvajalec naprave [27]. Njegova prednost zato ni v tem, da bi odpravil vse zaprtokodne dele, temveč v tem, da na razmeroma kakovostni in varnostno močni strojni podlagi zgradi strožje nastavljen odprtokoden sistemski sloj.

V dimenziji praktične uporabnosti je GrapheneOS na uradno podprtih napravah zelo močan, saj ohranja združljivost z aplikacijami za sistem Android in hkrati ne zahteva običajnega odpovedovanja ključnim funkcijam telefona. Njegova glavna omejitev je drugje: uporabnik mora sprejeti zelo ozek nabor podprtih naprav in s tem precej večjo vezanost na konkretno strojno platformo. GrapheneOS je zato dober primer rešitve, ki ni najbolj odprta v absolutnem smislu, vendar zelo premišljeno združuje odprtost zgornjih slojev, močan varnostni model in visoko praktično uporabnost.

5.1.3 /e/OS

Projekt /e/OS se od projektov LineageOS in GrapheneOS razlikuje predvsem po tem, da skuša odprtokodno mobilno rešitev približati širšemu krogu uporabnikov, ki jih zanimata zasebnost in manjša odvisnost od Googlovih storitev, ne želijo pa nujno tehnično zahtevnega sistema ali zelo ozkega nabora podprtih naprav [21, 20].

Njegova glavna ideja je različica sistema Android brez Googlovih storitev: odprtokoden sistemski temelj, nadomeščanje Googlovih storitev z alternativami ter bolj uporabniško usmerjen nabor privzetih aplikacij in spremljevalnih storitev. Dokumentacija za podprte naprave pri tem izrecno loči med graditvami tipa “Official” in “Community”; uradne graditve so po testiranju označene kot primerne za vsakodnevno rabo, obe vrsti pa prejemata posodobitve OTA [21]. To kaže, da projekt precej jasno cilja na vsakdanjo uporabnost in ne zgolj na tehnično demonstracijo odprtosti.

Po našem modelu je /e/OS zanimiv predvsem kot rešitev, ki skuša izboljšati odprtost in zasebnost na ravni sistemskih storitev, privzetih aplikacij in uporabniške izkušnje, ne da bi pri tem prekinila združljivost z običajno

rabo sistema Android. Tak pristop poveča praktično uporabnost, vendar hkrati ne odpravi enakih omejitev v spodnjih slojih kot pri drugih ROM-ih za sistem Android: tudi tu podpora napravam ostaja odvisna od dobaviteljskih komponent, strojne programske opreme in konkretne strojne osnove posameznega telefona.

Zaradi zelo širokega nabora podprtih naprav je profil sistema /e/OS nekoliko manj enoten kot pri projektu GrapheneOS. To pomeni, da se lahko dejanska kakovost podpore, hitrost posodobitev in obseg kompromisov med napravami precej razlikujejo. Vseeno pa je /e/OS pomemben primer zato, ker pokaže, da je mogoče odprtokodno mobilno rešitev usmeriti predvsem v praktično rabo in ne le v tehnično ali ideološko čistost. Med obravnavanimi ROM-i za sistem Android je zato /e/OS eden najbolj izrazitih primerov rešitve, ki daje prednost uporabniški dostopnosti.

5.1.4 Replicant

Projekt Replicant je v tej skupini poseben primer, ker kot osnovni cilj postavlja programsko svobodo in zato ne pristaja na številne pragmatične kompromise, ki jih sprejemajo drugi ROM-i za sistem Android. Predstavlja se kot popolnoma prosta distribucija sistema Android, ki ne vključuje in ne priporoča neproste programske opreme [46].

Z vidika idealnega modela iz prejšnjega poglavja je Replicant med ROM-i za sistem Android najbližje strožji interpretaciji odprtosti. Ravno zato pa zelo jasno pokaže, kako težko je v mobilnem svetu hkrati doseči visoko odprtost in visoko uporabnost. Ker zavračanje lastniških komponent močno omeji nabor uporabne strojne opreme, je seznam podprtih naprav zelo kratek in vezan predvsem na starejše modele [47]. To neposredno vpliva tudi na praktično uporabnost: že sama dostopnost primerne naprave je danes omejitev, poleg tega pa starejša strojna oprema težje zadosti sodobnim pričakovanjem glede zmogljivosti, avtonomije, kamer, omrežne podpore in aplikacijskega ekosistema.

Replicant je zato pomemben predvsem kot referenčni robni primer. Pokaže,

da je v okviru sistema Android mogoče zgraditi rešitev, ki programsko svobodo razume bistveno strožje kot projekti LineageOS, GrapheneOS ali /e/OS. Hkrati pa ista odločitev povzroči tako velike omejitve pri podpori napravam in pri vsakdanji rabi, da Replicant za večino uporabnikov danes težko predstavlja realno zamenjavo za sodoben telefon. Prav zato je za to diplomsko nalogo zelo uporaben: ne kot najbolj praktična rešitev, temveč kot dokaz, kako hitro se ob strožjem razumevanju odprtosti skrči nabor dejansko uporabnih naprav.

5.1.5 Vmesna primerjalna ugotovitev

Analiza alternativnih ROM-ov za sistem Android pokaže, da znotraj iste tehnične družine ne obstaja ena sama "najboljša" rešitev, temveč več različnih strategij. LineageOS predstavlja najbolj uravnotežen kompromis med odprtostjo in uporabnostjo. GrapheneOS kaže, kako daleč je mogoče priti z varnostno strogim pristopom, če projekt sprejme zelo ozek nabor strojne podpore. /e/OS daje največji poudarek vsakdanji uporabnosti in zmanjšanju odvisnosti od Googlovega ekosistema. Replicant pa predstavlja najbolj dosledno svobodnjaško smer, vendar za ceno zelo omejene praktične uporabnosti.

Skupna omejitev vseh teh rešitev je, da praviloma ne odpravijo temeljne odvisnosti od zaprtih komponent v spodnjih slojih mobilne naprave. Njihova največja prednost je zato predvsem v odprtosti uporabniškega prostora, večjem nadzoru nad sistemom, daljši življenjski dobi podprtih naprav in manjši vezanosti na proizvajalčev tovarniški sistem. To je pomemben korak naprej, vendar še ne pomeni, da tak telefon v celoti doseže ideal popolnoma odprtokodnega telefona, kot smo ga opredelili v prejšnjem poglavju.

5.2 Mobilne distribucije sistema GNU/Linux na obstoječi strojni opremi za sistem Android

Drugo pomembno skupino odprtokodnih mobilnih rešitev predstavljajo mobilne distribucije sistema GNU/Linux, namenjene delovanju na strojni opremi, ki je bila prvotno razvita za sistem Android. V primerjavi z alternativnimi ROM-i za sistem Android gre tu za drugačen pristop: namesto da bi uporabnik obdržal uporabniški prostor sistema Android in zamenjal predvsem sistemski sloj, skušajo ti projekti na obstoječo mobilno strojno opremo prenesti uporabniški prostor sistema GNU/Linux.

Tak pristop lahko poveča odprtost uporabniškega prostora, preglednost sistema in sorodnost z običajnimi distribucijami GNU/Linux. Hkrati pa ne odpravi samodejno omejitev v spodnjih slojih naprave. Ker je bila večina teh telefonov prvotno razvita za sistem Android, so mobilne distribucije sistema GNU/Linux pogosto še vedno odvisne od spodnjih slojev sistema Android, prilagojenih jeder, binarnih dodatkov in različnih združljivostnih slojev. Prav zato ta skupina rešitev dobro pokaže, da odprtost uporabniškega prostora še ni enaka odprtosti celotnega sistema.

V tej skupini obravnavamo Ubuntu Touch[5], Mobian[6], postmarketOS[7] in projekt Halium[8]. Izbrani so zato, ker predstavljajo štiri tesno povezane, vendar vsebinsko različne pristope: Ubuntu Touch kot uporabniško usmerjen mobilni sistem, Mobian kot mobilno distribucijo sistema GNU/Linux, tesno povezano z Debianom, postmarketOS kot širše zasnovano in bolj eksperimentalno mobilno distribucijo sistema GNU/Linux, Halium pa kot združljivostni temelj, ki skuša povezati svet naprav za sistem Android s projekti sistema GNU/Linux.

[5] `https://ubuntu-touch.io/`
[6] `https://mobian.org/`
[7] `https://postmarketos.org/`
[8] `https://halium.org/`

5.2.1 Ubuntu Touch

Projekt Ubuntu Touch je danes eden najpomembnejših predstavnikov mobilnih distribucij sistema GNU/Linux, ki skušajo odprtokodne mobilne rešitve približati tudi manj tehnično usmerjenim uporabnikom. Uradna dokumentacija projekta UBports ga opisuje kot mobilni operacijski sistem, osredotočen na enostavno uporabo, zasebnost in konvergenco [57, 56]. Že ta izhodišča kažejo, da projekt ne stavi predvsem na ideološko čistost ali eksperimentalnost, temveč na čim bolj zaokroženo uporabniško izkušnjo.

Za naš model je Ubuntu Touch zanimiv predvsem zato, ker poskuša na mobilni napravi ponuditi uporabniški prostor sistema GNU/Linux, ne da bi se pri tem povsem odpovedal praktični uporabnosti. Na ravni uporabniške izkušnje je projekt bližje klasičnemu razumevanju mobilnega operacijskega sistema kot številni bolj eksperimentalni projekti. Velik poudarek daje konsistentnemu uporabniškemu vmesniku in zaokroženi vsakdanji rabi. Pomemben poudarek je tudi ideja konvergence, torej približevanja mobilnega in namiznega načina uporabe v okviru istega sistema [56].

Vendar Ubuntu Touch hkrati zelo jasno pokaže omejitve te skupine rešitev. Dokumentacija za prenašanje sistema na nove naprave izrecno navaja, da je Ubuntu Touch mogoče prenesti na naprave, ki so bile prvotno dobavljene s sistemom Android, pri čemer je večina takih naprav do neke mere še vedno odvisna od lastniške programske opreme [58]. To pomeni, da Ubuntu Touch bistveno spremeni uporabniški prostor, ne odpravi pa nujno zaprtosti spodnjih slojev strojne podpore.

Po našem modelu bi zato Ubuntu Touch ocenili kot rešitev, ki je praviloma razmeroma močna na ravni odprtosti uporabniškega prostora, preglednosti sistema in nadzora nad privzetimi storitvami, nekoliko šibkejša pa na ravni gonilnikov, strojne programske opreme in celotne strojne neodvisnosti. V primerjavi z alternativnimi ROM-i za sistem Android je njegov položaj obraten: v nekaterih vidikih je bližje idealu odprtega sistema, vendar praviloma težje dosega enako raven aplikacijske združljivosti in splošne vsakdanje uporabnosti.

5.2.2 Mobian

Projekt Mobian predstavlja nekoliko drugačen pristop kot Ubuntu Touch, čeprav sodi v isto širšo skupino mobilnih distribucij sistema GNU/Linux. Na uradni strani se predstavlja kot izpeljanka Debiana za mobilne naprave, uradni blog projekta pa poudarja, da želi standardno distribucijo Debian povezati z mobilnimi prilagoditvami, pri čemer skuša čim več posebnih rešitev prenesti v izvorne projekte in s tem zmanjšati količino lastnih, projektno specifičnih sprememb [37, 38]. Že iz tega izhodišča je razvidno, da Mobian ne želi biti povsem ločen mobilni ekosistem, temveč čim bolj neposredna mobilna razširitev obstoječega ekosistema Debian.

Za naš model je Mobian zanimiv predvsem zato, ker je med obravnavanimi rešitvami eden najbližjih predstavi mobilnega sistema, ki ostaja tesno povezan z običajno distribucijo GNU/Linux. Tak pristop lahko poveča preglednost sistema, zmanjša količino projektno specifičnih prilagoditev in olajša razumevanje celotnega programskega sklada. Hkrati pa pomeni, da projekt manj stavi na strogo nadzorovano in poenoteno mobilno uporabniško izkušnjo kot Ubuntu Touch, temveč bolj na postopno prilagajanje obstoječega ekosistema GNU/Linux mobilni rabi.

Za to diplomsko nalogo je Mobian pomemben tudi zato, ker ni omejen le na namenske naprave z GNU/Linuxom, temveč podpira tudi nekatere telefone, prvotno razvite za sistem Android. Debianova dokumentacija je leta 2025 med uradno podprtimi napravami, ki so bile prvotno razvite za sistem Android, navajala OnePlus 6/6T in Pocophone F1 [19]. S tem Mobian jasno sodi v skupino rešitev, ki skušajo na obstoječo strojno opremo, prvotno razvito za sistem Android, prenesti uporabniški prostor sistema GNU/Linux.

Po našem modelu bi Mobian ocenili kot rešitev, ki je razmeroma močna na ravni odprtosti uporabniškega prostora, bližine običajnim distribucijam GNU/Linux in preglednosti programskega sklada, nekoliko manj enotna pa na ravni praktične uporabnosti. V primerjavi z Ubuntu Touch je praviloma manj osredotočen na čim bolj zaokroženo mobilno uporabniško izkušnjo, v primerjavi s postmarketOS pa deluje bolj konservativno in bližje klasični

logiki distribucije Debian. Prav zato je Mobian dober vmesni primer med uporabniško usmerjenim mobilnim sistemom in bolj eksperimentalnim prilagajanjem sistema GNU/Linux širokemu naboru naprav.

5.2.3 postmarketOS

Projekt postmarketOS predstavlja bolj izrazito eksperimentalno interpretacijo mobilnega sistema GNU/Linux kot Ubuntu Touch. Njegov cilj ni predvsem zgraditi čim bolj podoben nadomestek za običajen pametni telefon, temveč mobilno strojno opremo približati običajni logiki distribucij GNU/Linux in dolgoročnega vzdrževanja. Že uradna predstavitev projekta poudarja, da gre za razširitev distribucije Alpine Linux na pametne telefone [42].

Za naš model je postmarketOS posebej zanimiv zato, ker zelo jasno poudari razliko med široko podporo napravam in dejansko stopnjo uporabnosti. Seznam podprtih naprav je izredno velik; uradna dokumentacija je marca 2026 navajala 735 podprtih naprav [43]. Vendar sama podprtost še ne pomeni, da posamezna naprava ponuja enako stopnjo praktične uporabnosti. Pri številnih napravah to pomeni predvsem osnovno stopnjo podpore, medtem ko so posamezne funkcije, kot so mobilna omrežja, kamera, pospeševanje grafike ali upravljanje porabe, še vedno omejene.

Projekt postmarketOS je zato pomemben kot primer rešitve, ki močno razširi prostor eksperimentiranja in dolgoročnega oživljanja starejših naprav, vendar prav zaradi te širine težje zagotovi enotno uporabniško izkušnjo. Na ravni odprtosti uporabniškega prostora je pogosto zelo močan, na ravni praktične uporabnosti pa je profil med napravami bistveno bolj neenoten kot pri Ubuntu Touch ali pri alternativnih ROM-ih za sistem Android.

5.2.4 Vloga projekta Halium in sorodnih združljivostnih slojev

Projekt Halium sam po sebi ni končni mobilni operacijski sistem za vsakdanjo rabo, temveč združljivostni temelj za projekte, ki skušajo na naprave s prednameščenim sistemom Android prenesti uporabniški prostor sistema GNU/Linux. Uradna dokumentacija ga opredeli kot sodelovalni projekt za poenotenje abstrakcijske plasti strojne opreme za projekte, ki na mobilnih napravah s prednameščenim sistemom Android poganjajo GNU/Linux [29].

Za to diplomsko nalogo je Halium pomemben predvsem kot razlaga, zakaj mobilne distribucije sistema GNU/Linux na obstoječi strojni opremi pogosto ne morejo preprosto prekiniti vezi z okoljem sistema Android. Namesto popolne zamenjave spodnjih slojev sistema pogosto uporabljajo vmesne združljivostne rešitve, ki omogočijo uporabo obstoječih Androidovih komponent v drugačnem uporabniškem prostoru [29, 31, 49]. To pomeni, da je lahko uporabniški del sistema bistveno bolj odprt in bolj podoben običajnim distribucijam GNU/Linux, medtem ko spodnji del sklada ostaja tehnično in licenčno precej bolj hibriden.

5.2.5 Vmesna primerjalna ugotovitev

Analiza mobilnih distribucij sistema GNU/Linux na obstoječi strojni opremi za sistem Android pokaže drugačen profil kompromisov kot pri alternativnih ROM-ih za sistem Android. Če so slednji praviloma močnejši pri vsakdanji uporabnosti in združljivosti z mobilnim ekosistemom, so mobilne distribucije sistema GNU/Linux pogosto bolj zanimive z vidika odprtosti uporabniškega prostora, preglednosti sistema in dolgoročnega raziskovalnega potenciala.

Znotraj te skupine se kljub skupnemu izhodišču pokažejo tudi pomembne razlike med posameznimi projekti. Ubuntu Touch daje največji poudarek zaokroženi mobilni uporabniški izkušnji, Mobian ostaja najtesneje povezan z običajno distribucijo Debian, postmarketOS pa najizraziteje razširja prostor portanja in eksperimentiranja na velikem številu naprav. Halium medtem

pokaže, da številne takšne rešitve v spodnjih slojih še vedno ostajajo odvisne od okolja sistema Android in njegovih strojno-specifičnih komponent.

Prav ta skupina zato najbolj jasno pokaže, da je mobilna strojna oprema še vedno močno zaznamovana z dediščino sistema Android. Prehod na uporabniški prostor sistema GNU/Linux zato sam po sebi še ne pomeni popolnoma odprte naprave.

Literatura

[1] Abbas Acar in sod. "50 Shades of Support: A Device-Centric Analysis of Android Security Updates". V: *Proceedings of the Network and Distributed System Security Symposium (NDSS) 2024*. Velik empirični pregled uradnih zapisov posodobitev (2014–2023); pokaže razlike med OEM-i, zamude in omejeno trajanje varnostne podpore pri delu naprav. 2024. DOI: 10.14722/ndss.2024.24175. URL: https://www.ndss-symposium.org/wp-content/uploads/2024-175-paper.pdf (pridobljeno 14. 1. 2026).

[2] Android. *Android Enterprise Recommended Requirements*. Zahteve programa vključujejo tudi "Support current shipping release + one major OS upgrade" in zahteve glede varnostnih posodobitev. n.d. URL: https://www.android.com/enterprise/recommended/requirements/ (pridobljeno 14. 1. 2026).

[3] Android Developers. *Migration overview (Google Play Games Services)*. Vsebuje razlago, da je GMS Core/Google Play Services Googlov lastniški sloj na Androidu. Nov. 2025. URL: https://developer.android.com/games/pgs/migration_overview (pridobljeno 14. 1. 2026).

[4] Android Developers. *Overview of the Play Integrity API*. Okt. 2025. URL: https://developer.android.com/google/play/integrity/overview (pridobljeno 13. 1. 2026).

[5] Android Developers. *SafetyNet (deprecated): The SafetyNet Attestation API is deprecated and has been replaced by the Play Integrity API*. Opozorilo na strani navaja zamenjavo SafetyNet Attestation API s

Play Integrity API. Jan. 2026. URL: https://developer.android.com/privacy-and-security/safetynet (pridobljeno 13. 1. 2026).

[6] Android Open Source Project. *Android shared system image*. Opis Project Treble: ločitev strojno-specifičnega *vendor* dela in generičnega OS dela ter vmesnik VINTF. Dec. 2025. URL: https://source.android.com/docs/core/architecture/partitions/shared-system-image (pridobljeno 14. 1. 2026).

[7] Android Open Source Project. *Android Upgrade Invite*. Stran eksplicitno navaja, da Google Mobile Services (GMS) ni del AOSP in je na voljo le z licenco. Dec. 2025. URL: https://source.android.com/docs/core/ota/upgrade_invite (pridobljeno 14. 1. 2026).

[8] Android Open Source Project. *AOSP overview*. Opis Androida kot odprtokodnega programskega sklada za različne tipe naprav. 2025. URL: https://source.android.com/docs/setup/about (pridobljeno 19. 11. 2025).

[9] Android Open Source Project. *Interface versioning*. Opisuje mejo system.img/vendor.img pri Treble in zahtevo po verzioniranih vmesnikih. Avg. 2024. URL: https://source.android.com/docs/core/architecture/hidl/versioning (pridobljeno 14. 1. 2026).

[10] Android Open Source Project. *Mainline (modular system components)*. Opis modularnih posodobitev (Mainline) prek Google Play system update ali OTA mehanizmov partnerjev. Dec. 2025. URL: https://source.android.com/docs/core/ota/modular-system (pridobljeno 14. 1. 2026).

[11] Android Open Source Project. *Verified Boot*. Dokumentacija mehanizma Android Verified Boot (AVB). 2024. URL: https://source.android.com/docs/security/features/verifiedboot (pridobljeno 16. 2. 2026).

[12] *Apache License, Version 2.0*. Apache Software Foundation. URL: https://www.apache.org/licenses/LICENSE-2.0 (pridobljeno 4. 3. 2026).

[13] Apple. *App code signing process in iOS, iPadOS, tvOS, watchOS, and visionOS*. Apple Platform Security / Security Guide. Dec. 2024. URL: https://support.apple.com/guide/security/app-code-signing-process-sec7c917bf14/web (pridobljeno 14. 1. 2026).

[14] Apple Developer. *App Review Guidelines*. Nov. 2025. URL: https://developer.apple.com/app-store/review/guidelines/ (pridobljeno 14. 1. 2026).

[15] François Beaufort. *Media updates in Chrome 62*. Chrome for Developers; razlaga Widevine L1/L3 in povezavo z TEE. Sep. 2017. URL: https://developer.chrome.com/blog/media-updates-in-chrome-62 (pridobljeno 14. 1. 2026).

[16] Bitmovin. *Widevine Security Levels in Depth*. Sekundarni industrijski vir; opisuje tipično povezavo L1 → HD/UHD, L3 → SD. n.d. URL: https://developer.bitmovin.com/playback/docs/widevine-security-levels-in-web-video-playback (pridobljeno 14. 1. 2026).

[17] Conner Bradley. "Sustainable Security: Exploring Longevity Challenges and Solutions for IoT". V razdelku o dolgoživosti avtor izrecno omenja, da projekti, kot je LineageOS, podaljšujejo programsko podporo za naprave brez uradne podpore, npr. z portanjem novejših izdaj Androida na starejše telefone. Master's thesis. Ottawa, Ontario: Carleton University, 2023. URL: https://www.cisl.carleton.ca/~cbradley/data/papers/MastersThesis.pdf (pridobljeno 15. 1. 2026).

[18] Andrew Davis. "Cellular Baseband Security". Bachelor's thesis. Georgia Institute of Technology, 2012. URL: https://repository.gatech.edu/bitstreams/8cb73e2d-184d-41a3-8069-1d21d48ccc9a/download (pridobljeno 14. 1. 2026).

[19] Debian Wiki contributors. *Mobian/Devices*. Debianova dokumentacija za Mobian; navaja uradno podprte androidne naprave, med drugim OnePlus 6/6T in Pocophone F1. Sep. 2025. URL: https://wiki.debian.org/Mobian/Devices (pridobljeno 16. 3. 2026).

[20] e Foundation. *Different Build Types — /e/OS documentation*. Razlaga tipov gradenj Official, Community, Test, Unofficial in Custom. n.d. URL: `https://doc.e.foundation/build-status` (pridobljeno 16. 3. 2026).

[21] e Foundation. *Smartphone Selector — /e/OS documentation*. Seznam podprtih naprav in oznake Official, Community ter OTA. n.d. URL: `https://doc.e.foundation/devices` (pridobljeno 16. 3. 2026).

[22] Free Software Foundation. *What is Free Software?* Opredelitev proste programske opreme in štiri temeljne svoboščine. Jan. 2026. URL: `https://www.gnu.org/philosophy/free-sw.html` (pridobljeno 16. 2. 2026).

[23] *GNU General Public License, version 2*. Free Software Foundation. URL: `https://www.gnu.org/licenses/old-licenses/gpl-2.0.en.html` (pridobljeno 4. 3. 2026).

[24] Google for Developers. *Widevine DRM: Overview*. Okt. 2024. URL: `https://developers.google.com/widevine/drm/overview` (pridobljeno 14. 1. 2026).

[25] Google Play Console Help. *Use the Play Integrity API to detect risky interactions and fight abuse*. Uradna dokumentacija za razvijalce v Play Console; opisuje signale/verdict (npr. "genuine and certified Android device"). n.d. URL: `https://support.google.com/googleplay/android-developer/answer/11395166?hl=en` (pridobljeno 15. 1. 2026).

[26] GrapheneOS Project. *CLI install guide — Install — GrapheneOS*. Uradni vodič za namestitev na uradno podprte naprave. n.d. URL: `https://grapheneos.org/install/cli` (pridobljeno 16. 3. 2026).

[27] GrapheneOS Project. *Frequently Asked Questions — GrapheneOS*. Uradni FAQ; podprte naprave, razlogi za ozek nabor podpore in dolžina podpore. n.d. URL: `https://grapheneos.org/faq` (pridobljeno 16. 3. 2026).

[28] GrapheneOS Project. *GrapheneOS: the private and secure mobile OS*. Glej razdelek "About". 2026. URL: `https://grapheneos.org/` (pridobljeno 13. 1. 2026).

[29] Halium Project. *Halium – Halium documentation.* Uradni porting guide; opiše Halium kot projekt za poenotenje HAL za GNU/Linux sisteme na napravah s prednameščenim Androidom. 2023. URL: https://docs.halium.org/ (pridobljeno 14. 1. 2026).

[30] Grant Hernandez in sod. "FIRMWIRE: Transparent Dynamic Analysis for Cellular Baseband Firmware". V: *Network and Distributed System Security (NDSS) Symposium 2022.* 2022. DOI: 10.14722/ndss.2022.23136. URL: https://www.cise.ufl.edu/~butler/pubs/ndss22-firmwire.pdf (pridobljeno 14. 1. 2026).

[31] libhybris contributors. *libhybris: Load Android drivers from regular GNU/Linux processes.* README: libhybris omogoča nalaganje Android (bionic) knjižnic/gonilnikov znotraj procesov, ki uporabljajo npr. glibc ali musl. 2025. URL: https://github.com/libhybris/libhybris (pridobljeno 14. 1. 2026).

[32] LineageOS Project. *Devices — LineageOS Wiki.* Seznam uradno podprtih naprav LineageOS. 2026. URL: https://wiki.lineageos.org/devices/ (pridobljeno 16. 3. 2026).

[33] LineageOS Project. *Frequently Asked Questions — LineageOS Wiki.* Pojasnila o posodobitvah, Vendor SPL in drugih pogostih vprašanjih. 2026. URL: https://wiki.lineageos.org/faq (pridobljeno 16. 3. 2026).

[34] LineageOS Project. *Google apps — LineageOS Wiki.* Pojasni, da Google apps niso del LineageOS in da gre za lastniške aplikacije. 2026. URL: https://wiki.lineageos.org/gapps/ (pridobljeno 16. 3. 2026).

[35] LineageOS Project. *LineageOS – LineageOS Android Distribution.* Opis na strani navaja: "A free and open-source operating system for various devices, based on the Android mobile platform." URL: https://lineageos.org/ (pridobljeno 13. 1. 2026).

[36] Iliyan Malchev, Amith Dsouza in Veerendra Bhora. *Treble Plus One Equals Four.* Razlaga, kako arhitektura okoli Treble vpliva na stroške nadgradenj in podporne časovnice pri čipovjih (SoC) ter OEM-ih. Dec.

2020. URL: https://android-developers.googleblog.com/2020/12/treble-plus-one-equals-four.html (pridobljeno 14. 1. 2026).

[37] Mobian Project. *Mobian*. Uradna predstavitev projekta Mobian kot izpeljanke Debiana za mobilne naprave. n.d. URL: https://mobian.org/ (pridobljeno 16. 3. 2026).

[38] Mobian Project. *What is Mobian?* Uradni blog projekta; pojasni cilje projekta in poudari zmanjševanje projektno specifičnih sprememb z njihovim prenosom v izvorne projekte. Feb. 2021. URL: https://blog.mobian.org/posts/2021/02/18/what_is_mobian/ (pridobljeno 16. 3. 2026).

[39] Open Source Initiative. *The Open Source Definition*. Uradna definicija odprtokodne programske opreme (OSI). Feb. 2024. URL: https://opensource.org/osd (pridobljeno 16. 2. 2026).

[40] PINE64. *PinePhone*. Opis naprave PinePhone, poudari podporo za mainline Linux, strojna stikala za zasebnost ter osnovne specifikacije. n.d. URL: https://pine64.org/devices/pinephone/ (pridobljeno 14. 1. 2026).

[41] PINE64 Wiki. *PinePhone*. Dokumentacija in kontekst (izdaje, stanje programske opreme, namestitev, specifikacije, OS-ji). Nov. 2024. URL: https://wiki.pine64.org/wiki/PinePhone (pridobljeno 14. 1. 2026).

[42] postmarketOS Project. *About postmarketOS*. Opis ciljev in arhitekture postmarketOS kot razširitve distribucije Alpine Linux na pametne telefone. 2024. URL: https://wiki.postmarketos.org/wiki/About_postmarketOS (pridobljeno 19. 11. 2025).

[43] postmarketOS Project. *Devices — postmarketOS Wiki*. Seznam podprtih naprav; stran je marca 2026 navajala 735 podprtih naprav. Mar. 2026. URL: https://wiki.postmarketos.org/wiki/Devices (pridobljeno 16. 3. 2026).

[44] Purism. *Librem 5*. Uradna stran naprave Librem 5 (cilji, zasebnost, osnovne lastnosti in usmeritev v odprtokodnost). n.d. URL: https://puri.sm/products/librem-5/ (pridobljeno 14. 1. 2026).

[45] Purism Documentation. *Librem 5 – Documentation*. Tehnična dokumentacija za Librem 5 (programski sklad, uporaba, komponente). n.d. URL: https://docs.puri.sm/Librem_5/ (pridobljeno 14. 1. 2026).

[46] Replicant Project. *Replicant*. Uradna predstavitev projekta Replicant. n.d. URL: https://replicant.us/about.php (pridobljeno 16. 3. 2026).

[47] Replicant Project. *Supported devices*. Seznam podprtih in vzdrževanih naprav projekta Replicant. n.d. URL: https://replicant.us/supported-devices.php (pridobljeno 16. 3. 2026).

[48] Reproducible Builds. *Reproducible Builds*. Projekt in smernice za reproducibilne gradnje programskih paketov. n.d. URL: https://reproducible-builds.org/ (pridobljeno 16. 2. 2026).

[49] Sailfish OS Documentation. *Hardware Adaptation Development Kit – libhybris*. Opiše uporabo libhybris kot most med GNU libc uporabniškim prostorom in Android (bionic) komponentami pri strojnih prilagoditvah. 2024. URL: https://docs.sailfishos.org/Tools/Hardware_Adaptation_Development_Kit/ (pridobljeno 14. 1. 2026).

[50] StatCounter GlobalStats. *Mobile Operating System Market Share Worldwide*. Dostopno prek StatCounter GlobalStats, podatki za obdobje 2024–2025. 2025. URL: https://gs.statcounter.com/os-market-share/mobile/worldwide (pridobljeno 13. 1. 2026).

[51] Team Counterpoint. *Global Smartphone Sales Share by Operating System*. Counterpoint Research, Insight. Nov. 2025. URL: https://counterpointresearch.com/en/insights/global-smartphone-os-market-share (pridobljeno 13. 1. 2026).

[52] *The 3-Clause BSD License*. Open Source Initiative. URL: https://opensource.org/license/bsd-3-clause (pridobljeno 4. 3. 2026).

[53] The Linux Kernel Archives. *The Linux Kernel Archives*. Uradna stran izdaj jedra Linux in referenca za "mainline". n.d. URL: https://www.kernel.org/ (pridobljeno 16. 2. 2026).

[54] *The MIT License*. Open Source Initiative. URL: https://opensource.org/license/mit (pridobljeno 4. 3. 2026).

[55] Ken Thompson. "Reflections on Trusting Trust". V: *Communications of the ACM* 27.8 (1984), str. 761–763. DOI: 10.1145/358198.358210.

[56] UBports Community. *Convergence*. Uradna UBports dokumentacija; razlaga koncept konvergence (telefon ↔ namizje) in cilje uporabniške izkušnje. 2026. URL: https://docs.ubports.com/en/latest/humanguide/other-design-considerations/convergence.html (pridobljeno 15. 1. 2026).

[57] UBports Community. *Introduction – UBports documentation*. Uradna dokumentacija projekta UBports/Ubuntu Touch; opis ciljev projekta in kratka zgodovina. 2026. URL: https://docs.ubports.com/en/latest/about/introduction.html (pridobljeno 14. 1. 2026).

[58] UBports Community. *Introduction to porting — UBports documentation*. Pojasni, da je Ubuntu Touch mogoče prenesti na naprave, prvotno dobavljene s sistemom Android, in opozori na odvisnost od lastniške programske opreme. 2026. URL: https://docs.ubports.com/en/latest/porting/introduction/Intro.html (pridobljeno 16. 3. 2026).

[59] Widevine Help. *How to determine if Android device is security Level 1 or Level 3*. Google Support; vsebuje tudi status DEVICE_IS_PROVISIONED_SD_ONLY za Widevine L3. n.d. URL: https://support.google.com/widevine/answer/6072714?hl=en (pridobljeno 14. 1. 2026).